Lab 8: Mandatory Access Control, AppArmor and the Detection Signal

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

Discretionary access control (Unix permissions, Lab 6) lets the *owner* of a resource decide who may touch it, which means a compromised process running as an owner can hand its access away. **Mandatory access control** (MAC) removes that discretion: a system-wide policy constrains what a program may do regardless of the UID it runs as, so even a process running as root is boxed in. This lab has you author and load a minimal **AppArmor** profile for a service, read the denials it produces, cut it down to what the service needs, and then use those same logs the other way round: as a **detection signal**. This is where the course's intrusion-detection thread begins, and where **Lab report 3** is produced.

Safety. AppArmor confinement can stop a service from working, which is the point; nothing here locks *you* out. Snapshot `lab8-clean` before you start. The minimal profile you write here is small enough to read in full, so you load it straight into enforce mode.

Contents

- Learning Objectives
- Before the lab (at home, ~30-45 min)
- Part A: AppArmor on the system
- Part B: Author and load a minimal profile
- Part C: Read the denials, then tighten the profile
- Part D: The denial log as a detection signal
- Part E: The cross-platform view
- If it does not work
- Reflection
- References
- Deliverables: Lab report 3 (individual)

Learning Objectives

By completing this lab, students will be able to:

1. Explain how MAC differs from DAC, and why a system-wide policy contains a process the DAC model cannot.
2. Author a minimal AppArmor profile for a service, load it in enforce mode, and move it through complain and back to see what each mode reports.
3. Read AppArmor denial logs, explain what each one says about the profile, and cut a profile down to least privilege.
4. Use denial and audit logs as a **detection signal**: state what a defender watching them would see when the service is attacked, and what they would not.
5. Position AppArmor among the cross-platform MAC mechanisms (SELinux, FreeBSD jails, OpenBSD `pledge`/`unveil`, Windows integrity levels).

The detection work is the capstone's detection-artefact rehearsal.

What from this lab is assessed. This lab's write-up **is Lab report 3**, the one assessed deliverable of Block 3. It also collects the preparation questions of Labs 6, 7 and 8 (those of Labs 6 and 7 are in their handouts) and the reflections of all three labs. **Deliverables**, at the end of this handout, lists exactly what goes in.

Before the lab (at home, ~30-45 min)

Boot the VM, snapshot `lab8-clean`, re-read Lecture 8 if you have already had it. Confirm AppArmor and the audit tools are present (you build the confined target, a copy of `/bin/cat`, in Part B):

```
sudo aa-status | head                  # AppArmor is active on Ubuntu by default
                                       #   (aa-status needs root to read the profile
                                        ↪  set)
which aa-genprof aa-logprof aa-complain aa-enforce   # apparmor-utils
systemctl status auditd                # the audit daemon (for Part D)
```

What you are confining, and why nothing here can disturb your VM. You confine a small program that you build in the lab, not a real system service. AppArmor recognises it by its path. The program is a copy of `cat`, called `lab8svc`: a minimal "report reader" whose whole job is to read a file, so its allowed and denied reads are easy to watch. You create it in Exercise 2 with this command:

```
sudo cp /bin/cat /usr/local/bin/lab8svc
```

Preparation questions (submit with Lab report 3)

- **P1.** Think of one thing a process running as root did in Lab 6 or Lab 7. In two or three sentences: could a MAC policy on that program have refused it? Explain why or why not, what DAC checked in that case, and why it let root through.
- **P2.** Name a real service, other than the one in this lab, for which you would load a new AppArmor profile in *complain* mode first. In two or three sentences: why that service, and what risk does that choice accept?

Part A: AppArmor on the system

Exercise 1. See which AppArmor profiles the system loads, and read one written for a real service.

```
sudo aa-status                         # which profiles are loaded, in which mode
                                       #   (without sudo it exits 4: "not enough
                                        ↪  privilege")
ls -l /etc/apparmor.d/                 # the profiles that ship with the system
```

1a. **Read a real profile.** Nothing to write down here: this is to see what a profile looks like before you write your own in Part B. `rsyslogd`, the service that writes the system logs, runs confined by a profile that ships with Ubuntu:

```
ps -eo label,comm | grep rsyslogd                    # the running service, and
 ↪  its profile and mode
cat /etc/apparmor.d/usr.sbin.rsyslogd                 # the profile
```

Look for three kinds of line in it:

- **`capability ...`**: the capabilities the service may use (Lab 7). Any other capability is refused, even to root.
- **a path and letters**, such as `/etc/rsyslog.conf r,` or `/var/log/** rw,`: a file or a pattern of files, and what the service may do with it (`r` read, `w` write, `k` lock, `m` map as executable code). `**` matches any path below that folder.
- **`include`** or **`#include`**: a group of rules kept in another file. Careful: `#include` is a rule, not a comment. Any other line that starts with # is a comment.

The question that goes with this, about MAC and DAC, comes in Exercise 3, after you have seen a profile stop root.

Part B: Author and load a minimal profile

Exercise 2. Build the target, write a minimal profile for it by hand, and load it. You write it by hand so that you can see every rule; `aa-genprof` (below) is the assisted route.

First, the target: a copy of `cat`, the "report reader" whose job is to read a file.

```
sudo cp /bin/cat /usr/local/bin/lab8svc      # the "report reader": its job is to
↪  read a file
```

A profile is a file in `/etc/apparmor.d/`, by convention named after the program's path with `/` replaced by `.`: here, `/etc/apparmor.d/usr.local.bin.lab8svc`. Its two explicit rules allow the binary to map and read itself (`mr`) and to read one file, `/etc/hostname` (`r`); the `#include` adds more, as 2a shows. The command below writes the file with that content:

```
# a minimal profile: two explicit rules, plus what abstractions/base adds
sudo tee /etc/apparmor.d/usr.local.bin.lab8svc >/dev/null <<'PROF'
#include <tunables/global>
/usr/local/bin/lab8svc {
  #include <abstractions/base>
  /usr/local/bin/lab8svc mr,
  /etc/hostname r,
}
PROF
```

Load the profile, and check that it appears in the list of loaded profiles:

```
sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.lab8svc   # load it
sudo aa-status | grep lab8svc                # confirm it is loaded
```

`aa-status` lists the profile under the heading "profiles are in enforce mode"; the `grep` shows only the name.

The assisted route, for a real service, is `sudo aa-genprof /path/to/binary`: it generates a skeleton, you exercise the program in another terminal, and it scans the logs and asks you (A)llow/(D)eny per accessed path. `aa-complain` puts a profile in complain mode first: what it does not allow is logged but not stopped (an explicit `deny` rule is still enforced). You are doing the minimal version by hand here because a two-rule profile is something you can read in full.

2a. Your profile has two rules you can see: the binary may map itself, and it may read `/etc/hostname`. That is **not** everything it allows. `#include <abstractions/base>` is policy, not a comment: it adds what a program needs to run at all, such as reads of the loader's libraries, locale data and `/dev/urandom`, writes to `/dev/null` and the log socket, and part of `/proc`. The full list is in `/etc/apparmor.d/abstractions/base`.

- Record your profile.
- Predict: when you run `lab8svc` on `/etc/hostname` and on `/etc/shadow`, which read succeeds and which is denied?
- Run both with `sudo /usr/local/bin/lab8svc <file>` and show the two results (the terminal output or a screenshot). Did they match your prediction?
- Name one more file, besides `/etc/shadow`, that `cat` run with `sudo` reads normally and that this profile denies. Show it denied.

Only part of `/proc` is allowed. In `abstractions/base`, `@{PROC}` means `/proc` and `@{pid}` means a process number; `/proc/self` resolves to `/proc/<pid>` for the process that reads it.

- Read `abstractions/base` and list the `/proc` entries it grants.
- Name one `/proc` file this profile allows and one it denies.
- Show both with `sudo /usr/local/bin/lab8svc <file>`. Do the results match your answer to the previous point?

The difference between what you *think* a program may touch and what the policy really permits is also what an attacker looks for.

Part C: Read the denials, then tighten the profile

Exercise 3. The profile you loaded is already enforcing. You ran these two reads in 2a. Run them once more: this time you read what the log recorded for the denial, and then you do the least-privilege step.

```
sudo /usr/local/bin/lab8svc /etc/hostname
sudo /usr/local/bin/lab8svc /etc/shadow
echo "exit=$?"
```

You ran the reader as **root** (`sudo`), so every DAC check passed. Each denial is written to the audit log. Read it:

```
sudo grep -a 'apparmor="DENIED"' /var/log/audit/audit.log | tail
```

(`-a` makes `grep` treat the log as text. Without it, if the log contains a stray binary byte, which can happen after the VM is powered off abruptly or a snapshot is restored, `grep` prints only `binary file matches`.)

For a real service, you do not write the whole profile by hand. You use `sudo aa-logprof`, in a loop (you do not run it in this lab: a profile this small you write by hand):

1. Run the service under a minimal profile. By default, each access the profile does not allow is written to the audit log (a `deny` rule blocks silently; an `audit deny` rule blocks and logs).
2. Run `sudo aa-logprof`. It reads from the log what the profile did not allow and asks, for each one, whether to allow or deny it.
3. It writes your answers into the profile and reloads it.
4. Repeat until the service works under enforcement.

3a. Answer these questions:

- A process running as root, with its usual capabilities, passes the DAC checks, and was still denied here. What can an AppArmor profile forbid that root would otherwise be allowed to do?
- Why does that make MAC a different kind of control, and not only stronger file permissions?
- Name one thing DAC does well that MAC does not replace.
- Suppose a real service you are profiling is denied a file. Before you add a rule for it, how would you tell a file the service really needs from a file it was made to reach by an attacker?

Now the least-privilege step. **Remove the `/etc/hostname r` rule** from the profile (edit it with `sudo nano /etc/apparmor.d/usr.local.bin.lab8svc`) and reload it with `sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.lab8svc`.

- Run `sudo /usr/local/bin/lab8svc /etc/hostname` again and show the result. Is the read now denied?
- The program could use this rule, but a report reader's job does not need `/etc/hostname`. In one sentence: why remove it?

This is your final profile: `#include <abstractions/base>` and the one rule for the binary, without `/etc/hostname r`. Save a copy now, because 3d changes the file and the report asks for this version: `sudo cp /etc/apparmor.d/usr.local.bin.lab8svc ~/lab8-final-profile`. The copy is inside the VM: copy it out before you restore the snapshot at the end. Do not put the rule back: 3b, 3c and 3d use this profile.

3b. Name one operation your final profile prevents that an attacker who took over this service would want.

3c. **The other mode.** Put the profile into complain mode and run the *same forbidden read*:

```
sudo aa-complain /usr/local/bin/lab8svc
sudo /usr/local/bin/lab8svc /etc/shadow
echo "exit=$?"
```

Read what the log recorded this time:

```
sudo grep -a 'apparmor="ALLOWED"' /var/log/audit/audit.log | tail -2
```

Put the profile back into enforce mode and run the read again:

```
sudo aa-enforce /usr/local/bin/lab8svc
sudo /usr/local/bin/lab8svc /etc/shadow
```

In complain mode the forbidden read *happened* and was *recorded*. In enforce mode it was stopped and recorded. The word in the log tells you which: `ALLOWED` or `DENIED`.

- Record both outcomes and both log lines in your notes. For the `DENIED` line, run the `grep` from Exercise 3 again. (3c is not in the report, but 4b and Reflection 3 ask about it.)

Now suppose you must deploy a profile for a service you did not write.

- Which mode do you load it in first?
- What would make you change to the other mode?
- What risk does your choice accept?

There is no single right answer. A good answer says the risk of its own choice. Write it in your notes: 4b builds on it.

3d. **What the profile says about running another program.** Your profile has no execute rule at all. Here you find out what that means: you run the same test three times, with no rule, then with two different exec rules for `cat`.

`aa-exec -p <profile>` runs a command *confined by* that profile. For example, `sudo aa-exec -p /usr/local/bin/lab8svc -- /usr/bin/ls /` runs `ls` confined by your profile, as if it were `lab8svc`. `aa-exec` starts the command itself, so `ls` runs even though the profile has no exec rule; the profile's exec rules decide what that command may run *next*. Your profile does not allow reading `/`, so `ls` fails. Here you use a shell (`dash`) as the confined process, and `cat` as the program it tries to run. The `run` function below does that and prints the return code of the `cat` command:

```
run() { sudo aa-exec -p /usr/local/bin/lab8svc -- /usr/bin/dash \
          -c '/usr/bin/cat /etc/hostname 2>/dev/null; echo rc=$?'; }

run                                          # with no exec rule for cat at all
```

The return code tells you how far it got:

`rc`	meaning
`0`	`cat` ran and read the file
`1`	`cat` ran, but could not read the file (this is `cat`'s own failure code)
`126`	the shell found `cat` but was not allowed to execute it

An exec rule in a profile is a path followed by letters. The letters you need here:

- `r`: read the file;
- `m`: map it into memory as executable code;
- `ix`: **inherit-execute**: run it confined by the profile of the program that started it;
- `Px`: run it confined by a profile written for that program (here, a profile for `/usr/bin/cat`).

First, `Px`. Add the line `/usr/bin/cat Px,` to `/etc/apparmor.d/usr.local.bin.lab8svc` (with `sudo nano`), reload with `sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.lab8svc`, and call `run`.

Then your own line. Change that line so that the shell runs `cat` confined by the `lab8svc` profile itself. Reload, and call `run` a third time.

3d-i. Answer:

- Record the three `rc` values, and show the line you wrote.
- Explain what each part of your line does.
- Why does `Px` give the value it gives? (Is there a profile for `/usr/bin/cat` on this system? `sudo aa-status | grep cat` tells you.)

- What does the last value show about what `cat` inherits from `lab8svc` under your line?

3d-ii. **The path trap.** On Ubuntu, `/bin` is a symbolic link to `/usr/bin`, so `/bin/cat` and `/usr/bin/cat` are the same file. Most people type `/bin/cat`. This exercise shows what that means for a rule.

Your profile still has the line you wrote in 3d, the one that let `cat` run. Edit the profile and change **only the path** in that line, from `/usr/bin/cat` to `/bin/cat`, the one most people would type. Then reload it, and watch whether the parser complains:

```
sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.lab8svc
```

Now call `run` again:

```
run
```

- Record the `rc` value, and compare it with the one your line gave in 3d-i. Run `readlink -f /bin/cat` and compare its output with the path in your rule.
- Does it help to call `cat` by the path in your rule? Paste the whole `run` definition again with `/usr/bin/cat` changed to `/bin/cat`, call it, and record the `rc`.
- The same problem exists for a shell. Run `readlink -f /bin/sh` and show its output. A profile author writes a rule for `/bin/sh` to control which shell a service may start. What does that rule actually control, and what would the author have to write instead?

3d-iii. Suppose a profile allows broad reads **and** has an inherit-execute (`ix`) rule for a shell, and an attacker gets into that shell. The shell runs under the same profile. In one sentence, what does that shell give the attacker? You meet this situation again in Lab 12.

Part D: The denial log as a detection signal

The denials you used to fix the profile are also an **intrusion signal**: a confined process tried to do something its profile forbids.

Exercise 4. The forbidden read of `/etc/shadow` is what an abused service looks like. Now find that denial as a defender would, and notice how easy it is to look in the wrong place. First, make the forbidden read once more, so the denial is recent:

```
sudo /usr/local/bin/lab8svc /etc/shadow
```

Start with the two commands a defender usually reaches for first:

- `sudo ausearch -m AVC -ts recent` searches the audit log for AVC records (`-m AVC`: the record type for an access-control decision, such as a denial) from the last ten minutes (`-ts recent`);
- `sudo dmesg | grep -i apparmor` reads the kernel's message buffer and keeps the lines that mention AppArmor (`-i` ignores upper and lower case).

```
sudo ausearch -m AVC -ts recent
sudo dmesg | grep -i apparmor
```

Run both. Does either show your denial? Then read the audit log directly, and confirm the profile is in enforce mode:

```
sudo grep -a 'apparmor="DENIED"' /var/log/audit/audit.log | grep -a shadow | tail
sudo grep lab8svc /sys/kernel/security/apparmor/profiles
```

The last command prints the profile with its mode in brackets, for example `(enforce)`.

Why neither shows your denial: `ausearch` expects AVC records in SELinux's format, and AppArmor's lack the `tclass=` field it looks for, so it skips them. `dmesg` does not have it because, while `auditd` is running, the kernel normally hands audit records to `auditd` instead of printing them to its message buffer (we checked this on the arm64 course image: after `auditd` was stopped, the denial did appear in `dmesg`). Do not stop `auditd` in this lab: while it is stopped, new denials are not written to the audit log that 4a reads. `dmesg` may show `STATUS` lines from loading profiles, which are not your denial. A defender who trusted `ausearch` or `dmesg` alone would conclude ”no denials” while the service was being probed.

4a. **A defender's note.** Record one `DENIED` line for `/etc/shadow` from `/var/log/audit/audit.log`. Then write a short note about it as the defender who found it, by filling in this table:

field	your answer
Evidence: the log line, and the values of `apparmor`, `operation`, `profile`, `name`, `comm` and `pid`	
What was attempted: which process, which operation, on which target	
What stopped it: which profile, in which mode	
How you found it: which tools you tried, and which one worked	
What the line cannot tell you	

- Before an incident, how would you check that the tool you plan to rely on really shows this kind of event? Describe the test in one or two sentences.

4b. **Detection and prevention.** Think back to 3c: what happened to the forbidden read in each mode, and what the log said.

- What does a defender gain and lose with each mode?
- A good setup usually has both: a control that blocks, and a log that a person or a SIEM reads. Why are both needed?

The capstone's detection artefact builds on this answer.

Part E: The cross-platform view

Exercise 5. No hands-on, and **not done in the session**: write it in your own time and submit it with Lab report 3, together with the reflection. It is a short written comparison (this also appears in the lecture).

Compare AppArmor with four other mechanisms: **SELinux** (label-based, type enforcement), **FreeBSD jails**, **OpenBSD `pledge/unveil`** (a process restricting *itself*), and **Windows integrity levels / tokens**. Do it through the two cases you measured in this lab: `lab8svc` denied the read of `/etc/shadow` (Exercise 3), and the `/bin/cat` rule that stopped matching (3d-ii). Windows has no `/etc/shadow`: for Windows, take a file that ordinary users cannot read, such as the SAM database.

- For each of the four, in two or three sentences: how would it stop a process like `lab8svc` from reading `/etc/shadow`, and who writes that restriction (an administrator, or the program itself)?
- For each of the four: could the problem of 3d-ii (a rule written for one path that does not match the real file) happen there? Why, or why not?
- What idea do all five share?
- From your two cases: what does AppArmor's *path-based* model make easy, and what does it make fragile?

If it does not work

The first row is the point of Part D, not a fault.

What you see	What to do
After a denial, `ausearch -m AVC` and `dmesg` show no denial	On this image, while `auditd` runs, the denial is in `/var/log/audit/audit.log`, and the first two tools do not show it: `ausearch` skips it (it parses AVCs in SELinux's format, and AppArmor's have no `tclass=` field) and `dmesg` does not carry the denial, because the kernel hands it to `auditd`. `aa-notify` is not installed. Read the audit log directly. An empty result from your first tool is not evidence that nothing happened.

What you see	What to do
`grep` on the audit log prints only `binary file matches`	The log contains a stray binary byte (this can happen after the VM is powered off abruptly or a snapshot is restored). Add `-a` to `grep`, as the commands in this handout do, so it reads the log as text.

If none of these matches, read the error message in full first: it usually names the layer that refused, and that is most of the answer. Then ask your lab teacher, in the session or by email.

Reflection

Your reflection goes into Lab report 3, as an appendix, beside the reflections from the other labs in this block. They are assessed with the report: every one has to be there, each answering its questions with a position you argue from what you did in the lab.

In a short piece (200-300 words in all, about 100 for each question):

1. You used the denial log in two ways: to *build* a working profile and to *detect* an attack. What does it mean that prevention and detection use the same signal? What does a defender lose if they enforce the profile but never read the log?
2. AppArmor is path-based: it confines by file path, not by an inode label. Give one attack or maintenance situation where this is a weakness. What does SELinux's label model do differently?
3. Capstone link. You do not need the capstone brief for this: the three things it asks for are listed here. In the capstone, you describe each attack step as a detection artefact. For each step you give the action and its result, the record it left on the host, and whether a defender could tell an attack from a legitimate action from that record alone. Write these three for one step from this lab other than the denied read of 4a, for example the read of `/etc/shadow` in complain mode in 3c.

References

- `apparmor(7)`, `apparmor.d(5)`, `aa-genprof(8)`, `aa-logprof(8)`, `aa-complain`/`aa-enforce`.
- `auditd(8)`, `ausearch(8)`, the AVC (access-vector cache) message format.
- SELinux type enforcement; OpenBSD `pledge(2)`/`unveil(2)`; FreeBSD `jail(8)`; Windows integrity levels, for the cross-platform view.
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022, the MAC/SELinux material.

Deliverables: Lab report 3 (individual)

Lab report 3 is the write-up of this lab, submitted **individually**. It comprises: the profile you wrote in Part B and what it grants, with your prediction of which reads it allows and denies, the runs that check it, the extra denied file and the two `/proc` files (2a); your answers to every 3a question (the MAC-versus-DAC argument with this service as the example, one thing DAC does well that MAC does not replace, and how you would tell a file the service needs from one an attacker made it reach), **and** the least-privilege step: the rule you removed, the run that followed, and your one-sentence reason (Part C); one operation the final profile prevents that an attacker would have wanted (3b); the denial read as a **detection signal**, in the defender's note of 4a, with what it tells a defender and what it cannot, and your answer to the question after the table (Part D), and the detection-versus-prevention trade-off (4b); and the cross-platform comparison through your two cases (Part E). Include both profiles: the one you wrote in Part B and the final one from Exercise 3 (after the least-privilege step, before the exec rules of 3d) and the relevant log lines. Exercises

3c and 3d are done in the session and their answers are not part of the report, but 4b, Part E and Reflection 3 use what you saw in them. The final profile is the copy you saved in Exercise 3.

Marked on: technical correctness of the profile and iteration; explanation of what the denials told you; **security reasoning including the detection half**, which is new here and weighted; and preparation. It also collects the preparation questions from Labs 6, 7 and 8, and **their three reflections**, which go in as an appendix and are assessed with the report.

Due on Blackboard by **Sunday 22 November**, one week after Block 3 ends; the exact time is the one on the Blackboard assignment, and the late-report rule applies (one late report is free, up to seven days; see the assessment timeline).

Clean up when finished: unload and remove the profile and the target you built.

```
sudo apparmor_parser -R /etc/apparmor.d/usr.local.bin.lab8svc
sudo rm -f /etc/apparmor.d/usr.local.bin.lab8svc /usr/local/bin/lab8svc
```

Or simply restore `lab8-clean`. Nothing here was seeded into the base image, so the VM is back to stock either way.